

Aluno: Rodrigo Nunes Xavier
Matéria: Estrutura de Dados 2
Professora: Kadidja Valéria Reginaldo De Oliveira

ABB:

Organização de dados: Organizada em uma estrutura de dados, onde cada objeto (nó) possui uma chave e ponteiros para o pai, filho esquerdo e filho direito.

Regra ou propriedade: Para qualquer nó x , os nós na subárvore esquerda possuem chaves menores ou iguais a $x.chave$, e os nós na subárvore direita possuem chaves maiores ou iguais a $x.chave$.

Operação ou ajuste: A busca e a inserção descem da raiz às folhas comparando chaves. A ABB simples não faz ajustes estruturais (como rotações) de balanceamento após alterações, podendo degenerar no pior caso.

Aplicação: Usada para implementar dicionários dinâmicos que suportam operações como busca, mínimo, máximo, sucessor e predecessor, servindo de base para árvores avançadas.

Referência: [livro de referência](#)

AVL:

Organização de dados: Organizada em uma estrutura de dados, onde cada objeto (nó) possui uma chave e ponteiros para o pai, filho esquerdo e filho direito, além de que ele armazena o valor da altura.

Regra ou propriedade: A principal regra em uma árvore AVL é quando a diferença de altura entre as subárvores da esquerda e direita de um nó qualquer N , não é superior a 1.

Operação ou ajuste: Uma árvore AVL em sua operação de inserção, já realiza o balanceamento através de rotações (simples ou duplas, à esquerda ou direita) durante o processo de inserção e remoção justamente não deixando a diferença de altura ser maior que 1 entre as subárvores.

Aplicação: Ideal para sistemas em memória onde as operações de busca (leitura) são consideravelmente mais frequentes do que as inserções e remoções. É utilizada em aplicações que exigem consultas muito rápidas, como dicionários de verificação ortográfica, tabelas de roteamento em redes de computadores e índices de bancos de dados em memória.

Referência: [Site sobre AVL](#) e [Slide usado](#)

Rubro-Negra:

Organização de dados: Organizada em uma estrutura de dados, onde cada objeto (nó) possui uma chave e ponteiros para o pai e os filhos. Mas também tem um bit extra para as cores (Vermelha ou preta).

Regra ou propriedade: Para uma árvore binária de busca ser uma árvore rubro-negra tem

que seguir 5 regras:

- 1 - Todo nó é vermelho ou preto.
- 2 - A raiz é preta.
- 3- Toda folha é preta.
- 4 - Se um nó é vermelho, então os seus filhos são pretos.
- 5 - Para cada nó, todos os caminhos simples do nó até folhas descendentes contêm o mesmo número de nós pretos.

Operação ou ajuste: A busca, inserção e remoção ocorrem com garantia de tempo $O(\log n)$. Caso uma inserção quebre as regras das cores, a estrutura executa ajustes de recoloração e rotações (simples ou duplas) em tempo $O(1)$ para restaurar o balanceamento negro antes de concluir a operação.

Aplicação: Amplamente utilizada em sistemas e bibliotecas padrão que exigem garantia matemática de complexidade $O(\log n)$ no pior caso. É a estrutura base de mapas e conjuntos associativos em várias linguagens (como o `std::map` e `std::set` em C++, e o `TreeMap` em Java), além de ser utilizada no núcleo do sistema operacional Linux para organizar a fila de execução de processos (CFS - Completely Fair Scheduler).

Referência: [Link de referência](#)

B:

Organização de dados: Nós contêm múltiplas chaves, ponteiros para os filhos e os dados completos. As informações estão distribuídas por toda a árvore.

Regra ou propriedade: Os nós têm um grau mínimo t . Cada nó (exceto a raiz) possui entre $t - 1$ e $2t - 1$ chaves. Todas as folhas terminam obrigatoriamente na mesma profundidade.

Operação ou ajuste: A leitura e escrita manipulam páginas inteiras de memória. Ao exceder o limite de chaves, ocorre o *Split* (divisão), onde o nó é partido ao meio e a chave mediana sobe para o nó pai.

Aplicação: Sistemas de arquivos e bancos de dados específicos onde buscas por chaves únicas e isoladas precisam ser concluídas o mais rápido possível (pois o dado pode estar na raiz).

Referência: [livro de referência](#)

B+:

Organização de dados: Nós internos armazenam apenas chaves para roteamento. Os dados reais ficam restritos aos nós folha, que são interligados de forma encadeada (lista sequencial).

Regra ou propriedade: Segue as regras de grau e balanceamento perfeito da Árvore B, porém permite a duplicação de chaves (uma cópia no nó interno para guiar a busca e a chave real na folha com o dado).

Operação ou ajuste: Na operação de *Split* folha, o nó é dividido, mas a chave mediana é copiada para o pai em vez de apenas movida, garantindo que a base de folhas continue com todos os dados contíguos.

Aplicação: É o índice estrutural padrão da esmagadora maioria dos bancos de dados relacionais modernos, otimizada especificamente para acelerar buscas por intervalos e ordenação massiva de dados.

Referência: [livro de referência](#)

Árvore binária:

Organização de dados: Organizada hierarquicamente a partir de um nó raiz, onde cada nó pai possui ponteiros para, no máximo, dois nós filhos (o filho esquerdo e o filho direito), sem nenhuma exigência de que um lado seja maior ou menor que o outro.

Regra ou propriedade: O grau máximo de qualquer nó é obrigatoriamente 2 (podendo ter 0, 1 ou 2 filhos limitados a esquerdo e direito). Uma árvore binária tem arestas, uma aresta é a conexão entre 2 nós, uma árvore que tem N nós, possui $N - 1$ arestas. A profundidade de um nó é o comprimento do caminho da raiz até esse nó. A altura da árvore é o comprimento do caminho mais longo desde a raiz até qualquer nó foliar. O grau de um nó é o número de subárvores a ele associadas (ou seja, o número de filhos dele).

Operação ou ajuste: As operações básicas são inserção, remoção e consulta, acompanhadas obrigatoriamente dos algoritmos de percurso (pré-ordem, em-ordem e pós-ordem) para navegar pela estrutura. A árvore não realiza nenhum ajuste de balanceamento estrutural após sofrer alterações.

Aplicação: Aplicada em Árvores de Expressão (usadas por compiladores e calculadoras para resolver fórmulas matemáticas, onde o operador, como o '+', é o nó pai, e os números são os dois nós filhos) e na construção do algoritmo de compressão de dados Huffman.

Referência: [Link de referência](#)

Heap:

Organização de dados: A organização de dados em um Heap é bem coincidente com a árvore binária, considerando que o heap é uma árvore binária organizada entre nós, filhos à esquerda e à direita, mas é organizada em um vetor, onde os ponteiros para pai e filho não existem, eles são calculados por meios matemáticos.

Regra ou propriedade: É uma Árvore Binária Completa (todos os níveis preenchidos, exceto o último, da esquerda para a direita). Mantém a propriedade de Max-Heap (pai sempre maior ou igual aos filhos) ou Min-Heap (pai sempre menor ou igual aos filhos).

Operação ou ajuste: A inserção é feita na última folha (final do vetor) e a remoção ocorre sempre na raiz. O ajuste estrutural após essas alterações é o processo de *Heapify* (borbulhamento/cascata), que faz o valor subir ou descer na árvore através de trocas com o pai ou filhos até restaurar a regra do Max/Min-Heap.

Aplicação: Geralmente são muito utilizados em operações envolvendo grafos ou filas de prioridade, além de que é muito utilizado para consultas em arrays.

Referência: [Link de referência](#)

TRIE:

Organização de dados: Organizada como uma árvore *n-ária* onde os nós individuais geralmente não armazenam a chave inteira, mas sim um único caractere. Cada nó possui um conjunto de ponteiros para os próximos filhos (por exemplo, um vetor de 26 posições para o alfabeto) e um marcador booleano (como `isEndOfWord`) indicando se aquele caractere finaliza uma palavra válida no conjunto.

Regra ou propriedade: As chaves que possuem o mesmo prefixo compartilham o mesmo caminho inicial (nós ancestrais) a partir da raiz. A raiz representa uma string vazia, e o caminho traçado da raiz até um nó define a string associada a ele.

Operação ou ajuste: A busca e a inserção não comparam chaves inteiras, mas sim caractere por caractere, descendo de nível a cada letra. O desempenho não depende da quantidade total de palavras na árvore, mas apenas do tamanho L da palavra sendo processada, garantindo tempo $O(L)$. Diferente da AVL ou Rubro-Negra, a Trie não requer rotações ou ajustes de balanceamento estrutural após as alterações.

Aplicação: O uso clássico descrito na sua atividade é o índice que percorre letras sucessivas e compartilha o início de palavras de mesmo prefixo. Na prática, é a estrutura base por trás de ferramentas de autocompletar (como a barra de pesquisa do Google), corretores ortográficos, dicionários eficientes e roteamento de endereços IP.

Referência: [Link de referência](#)

Árvore Geral:

Organização de dados: Organizada de forma hierárquica a partir de um nó raiz, onde cada nó pode ter uma quantidade arbitrária (ilimitada) de nós filhos. Não há restrição de grau, ou seja, um nó pode ter zero, dezenas ou centenas de conexões dependentes.

Regra ou propriedade: Mantém uma relação estrita de ancestralidade (pai e filho), possuindo N nós e exatamente $N - 1$ arestas. Existe apenas um caminho único e simples da raiz até qualquer outro nó na estrutura, sem formação de ciclos, mas não há imposição de ordem de valores (como menores à esquerda ou maiores à direita).

Operação ou ajuste: Possui operações simples de inserção de filhos, remoção e algoritmos de percurso (geralmente em largura/nível ou em profundidade). Como não possui regras de limitação de altura ou ordenação matemática de chaves, a estrutura não requer e não realiza nenhum ajuste ou rotação de balanceamento estrutural após sofrer alterações.

Aplicação: Utilizada especificamente para representar hierarquias reais de mapeamento irrestrito. É a estrutura responsável por moldar o sistema de diretórios e arquivos de sistemas operacionais (como o Windows Explorer), organogramas corporativos e a árvore do DOM (*Document Object Model*) que os navegadores usam para renderizar tags HTML/XML na web.

Referência: [Link de referência](#)

Etapa 3:

1 - Trata-se de uma Árvore do tipo AVL. Ela não é permitida que os elementos fiquem com

altura maior que 1 dos outros.

Limitação de analogia: Na árvore AVL, as rotações são realizadas de forma lógica para garantir o balanceamento. Já na analogia, seria necessário organizar tudo fisicamente.

2 - Trata-se de uma árvore B. Na árvore B, realmente quando ocorre dela alcançar o limite de chaves, ela é dividida, também chamada de Split.

Limitação de analogia: Em uma árvore B, a divisão é alocada em blocos de memória e ponteiros, enquanto na analogia, seria necessário escrever novamente na folha dividida.

3 - Trata-se do Heap. Usando a propriedade Max-heap, garante-se que o maior valor fique com a maior prioridade.

Limitação de analogia: Em uma fila de pessoas, quando a primeira sai, as outras dão um passo à frente. Já em um Heap, o último elemento é posto na raíz.

4 - Trata-se do Trie. Com a propriedade de guardar os caracteres iniciais das Strings.

Limitação de analogia: Em um sumário, você encontra a palavra completa escrita. Já em um Trie, ele guarda o início da palavra e o resto é formado conforme o caminho é percorrido.

5 - Trata-se da Árvore Rubro-Negra. Uma estrutura que organiza uma árvore com base em cores além dos filhos, com regras que não ocorra desbalanceamento devido aos elementos em cor errada.

Limitação de analogia: Em uma árvore Rubro-Negra, os elementos não são realmente pintados, são apenas bits de memória para diferenciar os filhos e controlar o limite em cada lado.

6 - Trata-se de uma árvore B+. As chaves de busca guiam pelos nós internos, mas os registros completos estão todos no nível folha, que por sua vez possuem ponteiros laterais ligando-os sequencialmente para otimizar a leitura por intervalo

Limitação de analogia: Em um guia físico, ele te leva à página 10, caso queira ir para o próximo capítulo, você volta ao sumário e vai novamente ao próximo capítulo. Enquanto em uma árvore B+, você apenas desliza lateralmente aos próximas folhas, sem nunca retornar aos nós superiores.

7 - Trata-se de uma Árvore ABB. Uma estrutura de dados que exige que os valores menores sejam direcionados à esquerda, e os maiores direcionados à direita.

Limitação de analogia: Em uma analogia física, a ordem de chegada não importa, considerando que podem mudar a ordem a qualquer momento. Já em uma árvore ABB computacional, a ordem de inserção altera completamente a estrutura da árvore.